

Cours d'introduction à la programmation Python pour les Masters en maintenance électromécanique et en aérotechnique

Auteur : Manus AI, sous les ordres du Prof Papy NDUNGIDI

Introduction

Ce cours est spécialement conçu pour les étudiants en Master en alternance en gestion de la maintenance électromécanique et Master en science de l'ingénieur orientation aérotechnique, qui n'ont aucune expérience préalable en programmation. L'objectif est de vous fournir une base solide en programmation Python et de vous familiariser avec l'environnement interactif Jupyter Notebook, en mettant l'accent sur des applications concrètes et pertinentes pour votre domaine. Vous découvrirez comment Python peut devenir un outil indispensable pour l'analyse de données, l'automatisation, la maintenance prédictive et l'optimisation des processus.

La progression pédagogique sera graduelle, partant des concepts fondamentaux pour aboutir à des applications pratiques. Chaque module inclura des explications claires, des exemples de code interactifs et des exercices pour renforcer votre compréhension et vos compétences.

Objectifs généraux du cours

À la fin de ce cours, les étudiants seront capables de comprendre les concepts fondamentaux de la programmation avec Python, d'utiliser l'environnement Anaconda et Jupyter Notebook pour écrire, exécuter et documenter du code Python, de manipuler différents types de données et structures de contrôle, d'écrire des fonctions pour organiser et réutiliser leur code, de lire et écrire des données à partir de fichiers, d'appliquer les bases de Python à des problèmes simples liés à la maintenance électromécanique, et de déboguer et gérer les erreurs courantes dans leurs programmes.

Progression pédagogique et modules

Le cours est structuré en plusieurs modules, chacun abordant des concepts spécifiques avec une approche pratique. Cette structure vise à construire progressivement les compétences des étudiants, en s'assurant que chaque nouveau concept est bien ancré avant de passer au suivant, et en maintenant une forte pertinence avec leur domaine d'étude.

Module 1 : Introduction à l'environnement de travail (Anaconda et Jupyter Notebook)

Ce module est conçu pour vous familiariser avec les outils essentiels que nous utiliserons tout au long de ce cours : le langage de programmation **Python**, la plateforme de distribution **Anaconda**, et l'environnement de développement interactif **Jupyter Notebook**. Même sans expérience préalable en programmation, vous serez guidé pas à pas pour installer et configurer votre environnement de travail.

1.1 Qu'est-ce que Python et pourquoi est-il pertinent pour la maintenance électromécanique ?

Python est un langage de programmation de haut niveau, interprété, interactif et orienté objet. Il est réputé pour sa simplicité, sa lisibilité et sa polyvalence. Sa syntaxe claire et concise le rend particulièrement accessible aux débutants. [1]

Pour les professionnels de la maintenance électromécanique, Python offre des avantages considérables :

- **Analyse de données** : Python excelle dans le traitement et l'analyse de grandes quantités de données, ce qui est crucial pour l'interprétation des données de capteurs, la détection de tendances de défaillance, et l'optimisation des stratégies de maintenance prédictive. Des bibliothèques comme Pandas et NumPy sont des outils puissants pour cela.
- **Automatisation** : Il permet d'automatiser des tâches répétitives, telles que la génération de rapports, la gestion des plannings de maintenance, ou l'interaction avec des systèmes de gestion assistée par ordinateur (GMAO).
- **Machine Learning et Intelligence Artificielle** : Python est le langage de prédilection pour le développement d'algorithmes d'apprentissage automatique, qui peuvent être utilisés pour prédire les pannes d'équipements, optimiser les performances ou identifier des anomalies.
- **Connectivité** : Il peut interagir avec divers systèmes et équipements via des interfaces série, réseau ou des API, facilitant ainsi la collecte de données en temps réel.
- **Visualisation** : Des bibliothèques comme Matplotlib et Seaborn permettent de créer des graphiques et des tableaux de bord interactifs pour visualiser l'état des équipements et les performances de maintenance.

En somme, l'apprentissage de Python vous dotera d'un outil puissant pour moderniser et optimiser vos pratiques en maintenance électromécanique, en passant d'une approche réactive à une approche plus proactive et basée sur les données.

1.2 Présentation d'Anaconda : Installation et gestion des environnements

Anaconda est une distribution gratuite et open-source des langages Python et R pour la science des données et l'apprentissage automatique. Elle simplifie grandement l'installation et la gestion des paquets et des environnements Python. [2]

Pourquoi utiliser Anaconda ?

- 1 **Facilité d'installation** : Anaconda regroupe Python, Jupyter Notebook et des centaines de bibliothèques scientifiques populaires (NumPy, Pandas, Matplotlib, etc.) en un seul installateur, évitant ainsi les problèmes de dépendances.
- 2 **Gestion des environnements** : Il permet de créer des environnements virtuels isolés. Cela signifie que vous pouvez avoir différentes versions de Python ou de bibliothèques pour différents projets sans qu'elles n'interfèrent les unes avec les autres. C'est essentiel pour la gestion de projets complexes.

Installation d'Anaconda

Pour installer Anaconda, suivez les étapes suivantes :

- 3 **Téléchargement** : Rendez-vous sur le site officiel d'Anaconda (<https://www.anaconda.com/products/distribution>) et téléchargez l'installateur correspondant à votre système d'exploitation (Windows, macOS, Linux).
- 4 **Exécution de l'installateur** : Suivez les instructions de l'installateur. Il est généralement recommandé d'accepter les options par défaut, notamment l'ajout d'Anaconda à votre PATH (si l'option est proposée et recommandée).
- 5 **Vérification de l'installation** : Une fois l'installation terminée, ouvrez un terminal (ou l'invite de commande Anaconda Prompt sous Windows) et tapez `conda --version`. Si l'installation a réussi, la version de Conda s'affichera.

Gestion des environnements Conda

Voici quelques commandes de base pour gérer les environnements avec Conda :

- **Créer un nouvel environnement** :

```
conda create --name mon_environnement python=3.9
```

- (Remplacez `mon_environnement` par le nom souhaité et `3.9` par la version de Python si nécessaire).
- **Activer un environnement** :

```
conda activate mon_environnement
```

- **Désactiver un environnement** :

```
conda deactivate
```

- **Lister les environnements :**

```
conda env list
```

- **Installer un paquet dans l'environnement actif :**

```
conda install nom_du_paquet
```

- ou

```
pip install nom_du_paquet
```

- (Préférez conda install si le paquet est disponible via Conda, sinon utilisez pip).

1.3 Introduction à Jupyter Notebook

Jupyter Notebook est une application web open-source qui vous permet de créer et de partager des documents contenant du code en direct, des équations, des visualisations et du texte narratif. C'est un outil idéal pour l'apprentissage, l'exploration de données et la documentation de projets. [3]

Pourquoi Jupyter Notebook pour ce cours ?

- **Interactivité :** Vous pouvez exécuter du code cellule par cellule et voir les résultats immédiatement, ce qui est parfait pour expérimenter et comprendre les concepts.
- **Documentation intégrée :** Il permet de combiner du code Python avec des explications en texte (Markdown), des images et des graphiques, créant ainsi un document complet et auto-suffisant.
- **Facilité de partage :** Les notebooks Jupyter (.ipynb) sont facilement partageables et peuvent être ouverts et exécutés par d'autres utilisateurs.

Lancement de Jupyter Notebook

- 6 **Activer votre environnement (si vous en avez créé un) :**

```
conda activate mon_environnement
```

- 7 **Lancer Jupyter Notebook :**

```
jupyter notebook
```

- 8 Cette commande ouvrira une nouvelle fenêtre ou un nouvel onglet dans votre navigateur web par défaut, affichant l'interface de Jupyter.

L'interface de Jupyter Notebook

L'interface de Jupyter Notebook se compose de :

- **Explorateur de fichiers** : Une vue de vos fichiers et dossiers, à partir de laquelle vous pouvez naviguer, créer de nouveaux notebooks ou ouvrir des notebooks existants.
- **Notebooks** : Chaque notebook est composé de cellules.

Types de cellules

Un notebook Jupyter contient principalement deux types de cellules :

- 9 **Cellules de code** : Contiennent du code Python que vous pouvez exécuter. Le résultat de l'exécution (sortie, erreurs) s'affiche juste en dessous de la cellule.
- 10 **Cellules Markdown** : Contiennent du texte formaté à l'aide de la syntaxe Markdown. Vous pouvez y inclure des titres, des paragraphes, des listes, des liens, des images, etc. C'est là que vous rédigerez vos explications et commentaires.

Opérations de base dans un Notebook

- **Créer un nouveau notebook** : Dans l'explorateur de fichiers, cliquez sur New > Python 3 (ipykernel).
- **Exécuter une cellule** : Sélectionnez la cellule et cliquez sur le bouton Run (▶) dans la barre d'outils, ou utilisez le raccourci Shift + Enter.
- **Ajouter une cellule** : Cliquez sur le bouton + dans la barre d'outils.
- **Changer le type de cellule** : Utilisez le menu déroulant dans la barre d'outils pour passer de Code à Markdown et vice-versa.
- **Sauvegarder le notebook** : Cliquez sur l'icône de disquette ou utilisez Ctrl + S.

1.4 Premiers pas : Votre premier programme Python (Hello World!)

Il est de tradition, en programmation, de commencer par un programme simple qui affiche le message "Hello World!". Cela permet de vérifier que l'environnement est correctement configuré et de se familiariser avec l'exécution du code.

Dans une cellule de code Jupyter, tapez simplement :

```
print("Hello World!")
```

Exécutez la cellule (Shift + Enter). Vous devriez voir le résultat suivant apparaître juste en dessous :

```
Hello World!
```

Félicitations ! Vous avez exécuté votre premier programme Python. La fonction `print()` est utilisée pour afficher des informations à l'écran. Les guillemets (""") indiquent que "Hello World!" est une chaîne de caractères, c'est-à-dire du texte.

Références

[1] Python.org. *About Python*. Disponible sur : <https://www.python.org/about/> [2] Anaconda. *What is Anaconda?*. Disponible sur : <https://www.anaconda.com/what-is-anaconda> [3] Project Jupyter. *Jupyter Notebook*. Disponible sur : <https://jupyter.org/>

Module 2 : Les fondamentaux de Python

Ce module vous introduira aux concepts de base de la programmation en Python. Nous allons explorer les variables, les types de données fondamentaux, les opérateurs arithmétiques, de comparaison et logiques, ainsi que l'utilisation de fonctions mathématiques essentielles. Ces éléments constituent les piliers de tout programme Python.

2.1 Variables

Une **variable** est un nom symbolique qui fait référence à une valeur stockée en mémoire. C'est comme une boîte étiquetée dans laquelle vous pouvez ranger des informations. En Python, vous n'avez pas besoin de déclarer le type d'une variable avant de l'utiliser ; Python le détermine automatiquement. [1]

Déclaration et Affectation

Pour créer une variable, vous lui donnez un nom et vous lui affectez une valeur en utilisant le signe égal (=).

```
# Déclaration et affectation de variables
```

```
age = 30
nom = "Alice"
prix_unitaire = 15.75
est_actif = True

print(age)
print(nom)
print(prix_unitaire)
```

```
print(est_actif)
```

Conventions de nommage

Il existe des règles et des conventions pour nommer les variables en Python :

- Les noms de variables doivent commencer par une lettre (a-z, A-Z) ou un underscore (_).
- Ils ne peuvent contenir que des caractères alphanumériques (a-z, A-Z, 0-9) et des underscores.
- Ils sont sensibles à la casse (Age est différent de age).
- Il est recommandé d'utiliser des noms descriptifs et en minuscules, avec des underscores pour séparer les mots (convention snake case).
- Évitez d'utiliser les mots-clés réservés de Python (comme if, for, while, print, etc.).

```
# Exemples de bons et mauvais noms de variables
```

```
# Bonnes pratiques
nombre_capteurs = 5
temperature_moyenne = 25.5
```

```
# Mauvaises pratiques (à éviter)
# 1capteur = 10 # Commence par un chiffre
# temperature-moyenne = 20.0 # Contient un tiret
```

```
# class = "Maintenance" # 'class' est un mot-clé réservé
```

2.2 Types de données

Python gère automatiquement les types de données, mais il est essentiel de comprendre les types de base car ils déterminent les opérations que vous pouvez effectuer sur les valeurs. [2]

Nombres

- **Entiers (int)** : Nombres entiers positifs ou négatifs, sans décimales.
- **Flottants (float)** : Nombres décimaux.

```
# Exemples de nombres
```

```
quantite = 100    # int
poids = 75.5      # float
```

```
print(type(quantite))
```

```
print(type(poids))
```

Chaînes de caractères (str)

Les chaînes de caractères sont des séquences de caractères (texte) entourées de guillemets simples (') ou doubles (").

```
# Exemples de chaînes de caractères
```

```
message = "Bonjour le monde !"
modele_equipement = 'Moteur_XYZ_2025'
```

```
print(message)
```

```
print(modele_equipement)
```

Manipulation et formatage de chaînes

Vous pouvez concaténer (joindre) des chaînes, accéder à des caractères spécifiques ou formater des chaînes.

```
# Concaténation
```

```
marque = "Siemens"
modele = "S7-1500"
nom_complet = marque + " " + modele
print(nom_complet)
```

```
# Accès aux caractères (indexation)
premiere_lettre = nom_complet[0]
derniere_lettre = nom_complet[-1]
print(f"Première lettre : {premiere_lettre}, Dernière lettre : {derniere_lettre}")
```

```
# Slicing (extraction de sous-chaînes)
partie_modele = nom_complet[8:10] # Du 8ème caractère (inclus) au 10ème (exclu)
print(f"Partie du modèle : {partie_modele}")
```

```
# Formatage avec f-strings (Python 3.6+)
capacite = 500 # kW
message_info = f"L'équipement {modele_equipement} a une capacité de {capacite} kW."
```

```
print(message_info)
```

Booléens (bool)

Les booléens représentent des valeurs de vérité : True (vrai) ou False (faux). Ils sont fondamentaux pour les décisions dans les programmes.

```
# Exemples de booléens
```

```
est_en_marche = True  
alarme_active = False
```

```
print(type(est_en_marche))
```

```
print(alarme_active)
```

2.3 Opérateurs

Les opérateurs sont des symboles qui effectuent des opérations sur des valeurs et des variables. [3]

Opérateurs arithmétiques

Utilisés pour les calculs mathématiques.

Opérateur	Description	Exemple	Résultat
<u>+</u>	Addition	<u>10 + 3</u>	<u>13</u>
<u>=</u>	Soustraction	<u>10 - 3</u>	<u>7</u>
<u>*</u>	Multiplication	<u>10 * 3</u>	<u>30</u>
<u>/</u>	Division	<u>10 / 3</u>	<u>3.33...</u>
<u>%</u>	Modulo (reste)	<u>10 % 3</u>	<u>1</u>
<u>//</u>	Division entière	<u>10 // 3</u>	<u>3</u>

Opérateur	Description	Exemple	Résultat
<u>**</u>	Exposant	<u>10 ** 3</u>	<u>1000</u>

Exemples d'opérations arithmétiques

```
energie_consommee = 150 # kWh
duree_fonctionnement = 2.5 # heures

puissance_moyenne = energie_consommee / duree_fonctionnement
print(f"Puissance moyenne : {puissance_moyenne} kW")

nombre_cycles = 10
intervalle_maintenance = nombre_cycles % 3 # Maintenance tous les 3 cycles
```

```
print(f"Reste avant maintenance : {intervalle_maintenance} cycles")
```

Opérateurs de comparaison

Utilisés pour comparer deux valeurs ; ils retournent un booléen (True ou False).

Opérateur	Description	Exemple	Résultat
<u>==</u>	Égal à	<u>5 == 5</u>	<u>True</u>
<u>!=</u>	Différent de	<u>5 != 10</u>	<u>True</u>
<u><</u>	Inférieur à	<u>5 < 10</u>	<u>True</u>
<u>></u>	Supérieur à	<u>10 > 5</u>	<u>True</u>
<u><=</u>	Inférieur ou égal à	<u>5 <= 5</u>	<u>True</u>
<u>>=</u>	Supérieur ou égal à	<u>10 >= 5</u>	<u>True</u>

Exemples d'opérations de comparaison

```
temperature_actuelle = 85 # °C
temperature_seuil = 90 # °C
```

```

alerte_temperature = temperature_actuelle > temperature_seuil
print(f"Alerte température : {alerte_temperature}")

pression_actuelle = 10.2 # bar
pression_cible = 10.0 # bar

pression_ok = (pression_actuelle >= pression_cible - 0.5) and (pression_actuelle <= pression_cible + 0.5)

print(f"Pression dans la plage cible : {pression_ok}")

```

Opérateurs logiques

Utilisés pour combiner des expressions booléennes.

Opérateur	Description	Exemple	Résultat
<u>and</u>	Vrai si les deux opérandes sont vrais	<u>True and False</u>	<u>False</u>
<u>or</u>	Vrai si au moins un des opérandes est vrai	<u>True or False</u>	<u>True</u>
<u>not</u>	Inverse la valeur de vérité de l'opérande	<u>not True</u>	<u>False</u>

Exemples d'opérations logiques

```

capteur_1_defaillant = False
capteur_2_defaillant = True

alerte_generale = capteur_1_defaillant or capteur_2_defaillant
print(f"Alerte générale (un capteur défaillant) : {alerte_generale}")

systeme_critique = True
maintenance_requise = True

intervention_urgente = systeme_critique and maintenance_requise
print(f"Intervention urgente : {intervention_urgente}")

print(f"Le capteur 1 n'est PAS défaillant : {not capteur_1_defaillant}")

```

2.4 Fonctions mathématiques (module math)

Python dispose d'un module math intégré qui fournit des fonctions mathématiques avancées. Pour l'utiliser, vous devez d'abord l'importer. [4]

```
import math

# Exemples de fonctions mathématiques
rayon = 5
aire_cercle = math.pi * (rayon ** 2)
print(f"Aire du cercle : {aire_cercle:.2f}") # .2f pour formater à 2 décimales

valeur = 16
racine_carree = math.sqrt(valeur)
print(f"Racine carrée de {valeur} : {racine_carree}")

angle_degrees = 30
angle_radians = math.radians(angle_degrees)
sinus_angle = math.sin(angle_radians)
print(f"Sinus de {angle_degrees} degrés : {sinus_angle:.4f}")

logarithme_naturel = math.log(100)

print(f"Logarithme naturel de 100 : {logarithme_naturel:.2f}")
```

Ce module math est particulièrement utile pour les calculs d'ingénierie et d'analyse de données que vous pourriez rencontrer en maintenance électromécanique.

Références

- [1] Python.org. *The Python Tutorial - 3. An Informal Introduction to Python*. Disponible sur : <https://docs.python.org/3/tutorial/introduction.html#first-steps-towards-programming> [2] Python.org. *Built-in Types*. Disponible sur : <https://docs.python.org/3/library/stdtypes.html> [3] W3Schools. *Python Operators*. Disponible sur : https://www.w3schools.com/python/python_operators.asp [4] Python.org. *math* — *Mathematical functions*. Disponible sur : <https://docs.python.org/3/library/math.html>

Module 3 : Structures de données essentielles

Dans ce module, nous allons explorer les structures de données fondamentales en Python. Ces structures vous permettent d'organiser et de manipuler des collections de données de manière efficace. Comprendre comment utiliser les listes, les tuples, les dictionnaires et les

ensembles est crucial pour gérer des informations plus complexes que de simples variables individuelles.

3.1 Listes (list)

Une **liste** est une collection ordonnée et modifiable d'éléments. Les éléments d'une liste peuvent être de types différents (nombres, chaînes de caractères, même d'autres listes). Les listes sont définies en plaçant les éléments entre crochets [], séparés par des virgules. [1]

Création et Accès aux éléments

```
# Création d'une liste d'équipements

equipements = ["Moteur_A", "Pompe_B", "Compresseur_C", "Vanne_D"]
print(f"Liste des équipements : {equipements}")

# Création d'une liste de températures de capteurs
temperatures = [25.3, 26.1, 24.9, 27.0, 25.5]
print(f"Températures relevées : {temperatures}")

# Accès aux éléments par index (l'index commence à 0)
premier_equipement = equipements[0]
dernier_equipement = equipements[-1] # Accès au dernier élément
print(f"Premier équipement : {premier_equipement}")
print(f"Dernier équipement : {dernier_equipement}")

# Slicing (extraction d'une sous-liste)
premiers_deux_equipements = equipements[0:2] # Éléments d'index 0 et 1

print(f"Les deux premiers équipements : {premiers_deux_equipements}")
```

Modification, Ajout et Suppression d'éléments

Les listes sont mutables, ce qui signifie que vous pouvez modifier leurs éléments après leur création.

```
# Modification d'un élément

equipements[1] = "Pompe_E"
print(f"Liste après modification : {equipements}")

# Ajout d'un élément à la fin (append)
equipements.append("Ventilateur_F")
print(f"Liste après ajout : {equipements}")
```

```
# Insertion d'un élément à un index spécifique (insert)
equipements.insert(2, "Capteur_G")
print(f"Liste après insertion : {equipements}")

# Suppression d'un élément par sa valeur (remove)
equipements.remove("Moteur_A")
print(f"Liste après suppression par valeur : {equipements}")

# Suppression d'un élément par son index (pop)
equipement_supprime = equipements.pop(0) # Supprime et retourne l'élément à l'index 0
print(f"Équipement supprimé : {equipement_supprime}")

print(f"Liste après suppression par index : {equipements}")
```

Méthodes utiles pour les listes

Méthode	Description	Exemple
<u>len()</u>	Retourne le nombre d'éléments dans la liste	<u>len(equipements)</u>
<u>sort()</u>	Trie la liste en place (modifie la liste originale)	<u>temperatures.sort()</u>
<u>sorted()</u>	Retourne une nouvelle liste triée	<u>sorted(temperatures)</u>
<u>count()</u>	Compte le nombre d'occurrences d'un élément	<u>equipements.count("Pompe E")</u>
<u>index()</u>	Retourne l'index de la première occurrence	<u>equipements.index("Compresseur C")</u>

```
# Exemples de méthodes de liste
```

```
liste_numerique = [3, 1, 4, 1, 5, 9, 2]
print(f"Liste originale : {liste_numerique}")

liste_numerique.sort() # Trie la liste en place
print(f"Liste triée : {liste_numerique}")

nombre_elements = len(liste_numerique)
print(f"Nombre d'éléments : {nombre_elements}")

compte_un = liste_numerique.count(1)

print(f"Le nombre 1 apparaît {compte_un} fois.")
```

3.2 Tuples (tuple)

Un **tuple** est une collection ordonnée et **immuable** d'éléments. Une fois créé, un tuple ne peut pas être modifié (ajout, suppression ou modification d'éléments). Les tuples sont définis en plaçant les éléments entre parenthèses `()`, séparés par des virgules. [2]

Création et Accès

```
# Création d'un tuple représentant les coordonnées d'un point (x, y, z)

coordonnees_capteur = (10.5, 20.1, 5.0)
print(f"Coordonnées du capteur : {coordonnees_capteur}")

# Création d'un tuple pour les paramètres d'un moteur (puissance, tension, courant)
parametres_moteur = ("50kW", "400V", "80A")
print(f"Paramètres moteur : {parametres_moteur}")

# Accès aux éléments (comme les listes, par index)
puissance = parametres_moteur[0]
print(f"Puissance du moteur : {puissance}")

# Les tuples peuvent contenir des types de données mixtes
info_equipement = ("Pompe_A", 2023, True, 1500.0)

print(f"Informations équipement : {info_equipement}")
```

Immutabilité

```
# Tentative de modification d'un tuple (cela générera une erreur)

# coordonnees_capteur[0] = 11.0 # TypeError: 'tuple' object does not support item assignment
```

Les tuples sont souvent utilisés pour des collections de données qui ne doivent pas changer, comme des coordonnées, des enregistrements de base de données, ou des valeurs de configuration.

3.3 Dictionnaires (dict)

Un **dictionnaire** est une collection non ordonnée de paires clé-valeur. Chaque clé doit être unique et immuable (comme une chaîne de caractères ou un nombre), et elle est associée à une valeur. Les dictionnaires sont définis en plaçant les paires clé-valeur entre accolades `{}`, séparées par des virgules. [3]

Création et Accès par clé

```
# Création d'un dictionnaire pour les informations d'un capteur

capteur_info = {
    "ID": "C001",
    "Type": "Température",
    "Localisation": "Salle des machines",
    "Valeur_actuelle": 35.7,
    "Unite": "°C"
}
print(f"Informations du capteur : {capteur_info}")

# Accès à une valeur par sa clé
type_capteur = capteur_info["Type"]
print(f"Type de capteur : {type_capteur}")

# Accès sécurisé avec .get() (retourne None si la clé n'existe pas, ou une valeur par défaut)
statut = capteur_info.get("Statut", "Inconnu")

print(f"Statut du capteur : {statut}")
```

Ajout, Modification et Parcours

```
# Ajout d'une nouvelle paire clé-valeur

capteur_info["Seuil_alerte"] = 40.0
print(f"Après ajout du seuil : {capteur_info}")

# Modification d'une valeur existante
capteur_info["Valeur_actuelle"] = 36.1
print(f"Après mise à jour de la valeur : {capteur_info}")

# Suppression d'une paire clé-valeur
del capteur_info["Unite"]
print(f"Après suppression de l'unité : {capteur_info}")

# Parcours des clés, valeurs ou paires clé-valeur
print("Clés du dictionnaire :")
for cle in capteur_info.keys():
    print(cle)

print("Valeurs du dictionnaire : ")
for valeur in capteur_info.values():
    print(valeur)
```

```
print("Paires clé-valeur du dictionnaire :")
for cle, valeur in capteur_info.items():

    print(f"{cle}: {valeur}")
```

Les dictionnaires sont extrêmement utiles pour représenter des objets ou des enregistrements où chaque propriété a un nom unique.

3.4 Ensembles (set)

Un **ensemble** est une collection non ordonnée d'éléments **uniques**. Les ensembles sont utiles pour stocker des collections d'éléments où l'ordre n'a pas d'importance et où les doublons ne sont pas autorisés. Ils sont définis en plaçant les éléments entre accolades {}, séparés par des virgules, ou en utilisant la fonction `set()`. [4]

Création et Opérations ensemblistes

```
# Création d'un ensemble de codes d'erreur uniques

codes_erreur_jour1 = {"E001", "E005", "E002", "E001"}
print(f"Codes d'erreur jour 1 : {codes_erreur_jour1}") # Notez que E001 n'apparaît qu'une seule fois

codes_erreur_jour2 = {"E002", "E003", "E006"}
print(f"Codes d'erreur jour 2 : {codes_erreur_jour2}")

# Union : tous les codes d'erreur uniques sur les deux jours
all_codes = codes_erreur_jour1.union(codes_erreur_jour2)
print(f"Tous les codes d'erreur uniques : {all_codes}")

# Intersection : codes d'erreur communs aux deux jours
codes_communs = codes_erreur_jour1.intersection(codes_erreur_jour2)
print(f"Codes d'erreur communs : {codes_communs}")

# Différence : codes d'erreur du jour 1 non présents au jour 2
codes_specifiques_jour1 = codes_erreur_jour1.difference(codes_erreur_jour2)
print(f"Codes spécifiques au jour 1 : {codes_specifiques_jour1}")

# Vérifier l'appartenance
if "E005" in codes_erreur_jour1:

    print("E005 est présent dans les codes d'erreur du jour 1.")
```

Les ensembles sont particulièrement efficaces pour des opérations comme la vérification rapide de l'appartenance d'un élément ou l'élimination des doublons.

Références

[1] Python.org. *The Python Tutorial - 5. Data Structures - 5.1. More on Lists*. Disponible sur : <https://docs.python.org/3/tutorial/datastructures.html#more-on-lists> [2] Python.org. *The Python Tutorial - 5. Data Structures - 5.3. Tuples and Sequences*. Disponible sur : <https://docs.python.org/3/tutorial/datastructures.html#tuples-and-sequences> [3] Python.org. *The Python Tutorial - 5. Data Structures - 5.5. Dictionaries*. Disponible sur : <https://docs.python.org/3/tutorial/datastructures.html#dictionaries> [4] Python.org. *The Python Tutorial - 5. Data Structures - 5.4. Sets*. Disponible sur : <https://docs.python.org/3/tutorial/datastructures.html#sets>

Module 4 : Contrôle du flux d'exécution

Dans ce module, nous allons apprendre à contrôler l'ordre dans lequel les instructions de votre programme sont exécutées. C'est ce qu'on appelle le **contrôle du flux**. Cela vous permettra de prendre des décisions dans votre code et de répéter des actions, ce qui est fondamental pour créer des programmes dynamiques et utiles. Nous aborderons les structures conditionnelles (if, elif, else) et les boucles (for, while).

4.1 Conditions (if, elif, else)

Les structures conditionnelles permettent à votre programme de prendre des décisions. Elles exécutent un bloc de code si une certaine condition est vraie, et un autre bloc si la condition est fausse. [1]

La déclaration if

La déclaration if est la plus simple. Elle exécute un bloc de code si la condition spécifiée est True.

```
temperature_capteur = 75 # °C

seuil_alerte = 70 # °C

if temperature_capteur > seuil_alerte:
    print("Alerte : Température du capteur supérieure au seuil !")
    print("Vérifier l'équipement immédiatement.")

print("Vérification de la température terminée.")
```

Notez l'indentation (les espaces au début des lignes) après la déclaration `if`. En Python, l'indentation est cruciale ; elle définit les blocs de code. Un bloc de code indenté est exécuté uniquement si la condition est vraie.

La déclaration if-else

La déclaration if-else permet d'exécuter un bloc de code si la condition est vraie, et un autre bloc si la condition est fausse.

```
pression_huile = 3.5 # bar

pression_min_requise = 3.0 # bar

if pression_huile < pression_min_requise:
    print("Alerte : Pression d'huile insuffisante !")
    print("Arrêt d'urgence de la machine.")
else:

    print("Pression d'huile normale. Fonctionnement continu.")
```

La déclaration if-elif-else

Lorsque vous avez plusieurs conditions à vérifier, vous pouvez utiliser elif (abréviation de "else if"). Python vérifiera les conditions dans l'ordre, et exécutera le premier bloc if ou elif dont la condition est vraie. Si aucune condition n'est vraie, le bloc else (s'il existe) sera exécuté.

```
niveau_usure_roulement = 0.6 # entre 0 (neuf) et 1 (usé)

if niveau_usure_roulement >= 0.9:
    print("Remplacement du roulement requis de toute urgence.")
elif niveau_usure_roulement >= 0.7:
    print("Planifier le remplacement du roulement à court terme.")
elif niveau_usure_roulement >= 0.5:
    print("Surveiller l'usure du roulement. Remplacement à moyen terme.")
else:

    print("Roulement en bon état. Surveillance régulière.")
```

4.2 Boucles for

Les boucles for sont utilisées pour itérer sur une séquence (comme une liste, un tuple, une chaîne de caractères ou un range) ou d'autres objets itérables. Elles permettent d'exécuter un bloc de code un nombre de fois connu, une fois pour chaque élément de la séquence. [2]

Itération sur une liste

```
equipements_a_verifier = ["Moteur_1", "Pompe_A", "Compresseur_B"]

print("Début de la vérification des équipements :")
for equipement in equipements_a_verifier:
    print(f"- Vérification de l'équipement : {equipement}")
    # Ici, vous pourriez ajouter du code pour interroger l'état de l'équipement

print("Vérification des équipements terminée.")
```

Itération avec range()

La fonction range() génère une séquence de nombres. Elle est souvent utilisée pour répéter une action un certain nombre de fois.

- range(n) : génère des nombres de 0 à n-1.
- range(start, stop) : génère des nombres de start à stop-1.
- range(start, stop, step) : génère des nombres de start à stop-1 avec un pas de step.

```
# Répéter une action 5 fois

print("Exécution de 5 cycles de test :")
for i in range(5):
    print(f"Cycle de test numéro {i + 1}")

# Itérer sur les indices d'une liste
capteurs_defaillants = ["Capteur_T1", "Capteur_P2", "Capteur_V3"]
for i in range(len(capteurs_defaillants)):

    print(f"Le capteur {capteurs_defaillants[i]} à l'index {i} est défaillant.")
```

Itération sur une chaîne de caractères

```
code_erreur = "E007"
```

```
print("Caractères du code d'erreur :")
for caractere in code_erreur:

    print(caractere)
```

4.3 Boucles while

Les boucles while sont utilisées pour exécuter un bloc de code de manière répétée tant qu'une condition est vraie. Elles sont utiles lorsque le nombre de répétitions n'est pas connu à l'avance. [3]

```
compteur_tentatives = 0

max_tentatives = 3

while compteur_tentatives < max_tentatives:
    print(f"Tentative de connexion au système... ({compteur_tentatives + 1}/{max_tentatives})")
    # Simuler une tentative de connexion
    connexion_reussie = False # Supposons que la connexion échoue pour l'exemple

    if connexion_reussie:
        print("Connexion établie !")
        break # Sortir de la boucle si la connexion réussit
    else:
        print("Échec de la connexion.")
        compteur_tentatives += 1 # Incrémenter le compteur

if compteur_tentatives == max_tentatives:

    print("Nombre maximal de tentatives atteint. Abandon.")
```

Il est crucial de s'assurer que la condition de la boucle while finira par devenir fausse, sinon vous créerez une **boucle infinie** qui bloquera votre programme.

4.4 Instructions de contrôle de boucle (break, continue)

Ces instructions vous donnent plus de contrôle sur l'exécution des boucles. [4]

break

L'instruction break permet de sortir immédiatement de la boucle la plus proche (qu'elle soit for ou while). L'exécution du programme reprendra à l'instruction qui suit la boucle.

```
liste_capteurs = ["Capteur_T1", "Capteur_P2", "Capteur_V3", "Capteur_D4", "Capteur_H5"]
```

```
capteur_critique_a_trouver = "Capteur_V3"
```

```
for capteur in liste_capteurs:
    if capteur == capteur_critique_a_trouver:
        print(f"Capteur critique {capteur} trouvé ! Arrêt de la recherche.")
        break # Sort de la boucle for
    print(f"Vérification du capteur : {capteur}")
```

```
print("Processus de recherche terminé.")
```

continue

L'instruction continue permet de sauter le reste du code dans l'itération actuelle de la boucle et de passer à l'itération suivante.

```
valeurs_capteur = [10, 12, -5, 15, 0, 18, -2]
```

```
print("Traitement des valeurs positives du capteur :")
for valeur in valeurs_capteur:
    if valeur <= 0:
        print(f"Valeur non valide ({valeur}), passage à la suivante.")
        continue # Passe à l'itération suivante sans exécuter le code ci-dessous
    print(f"Traitement de la valeur positive : {valeur}")
```

```
# Ici, vous pourriez ajouter du code pour analyser la valeur positive
```

Références

[1] Python.org. *The Python Tutorial - 4. More Control Flow Tools - 4.1. if Statements*. Disponible sur : <https://docs.python.org/3/tutorial/controlflow.html#if-statements> [2]
Python.org. *The Python Tutorial - 4. More Control Flow Tools - 4.2. for Statements*. Disponible sur : <https://docs.python.org/3/tutorial/controlflow.html#for-statements> [3]
Python.org. *The Python Tutorial - 4. More Control Flow Tools - 4.3. The range() Function*. Disponible sur : <https://docs.python.org/3/tutorial/controlflow.html#the-range-function> [4]
Python.org. *The Python Tutorial - 4. More Control Flow Tools - 4.4. break and continue Statements, and else Clauses on Loops*. Disponible sur : <https://docs.python.org/3/tutorial/controlflow.html#break-and-continue-statements-and-else-clauses-on-loops>

Module 5 : Fonctions et modularité

Dans ce module, nous allons aborder un concept fondamental en programmation : les **fonctions**. Les fonctions sont des blocs de code réutilisables qui effectuent une tâche spécifique. Elles permettent d'organiser votre code, de le rendre plus lisible, plus facile à maintenir et d'éviter la répétition. La modularité est essentielle pour écrire des programmes efficaces et complexes.

5.1 Définition de fonctions

Une fonction est définie en Python à l'aide du mot-clé `def`, suivi du nom de la fonction, de parenthèses `()` qui peuvent contenir des paramètres, et d'un double point `:`. Le corps de la fonction est indenté. [1]

Syntaxe de base

```
def nom_de_la_fonction(parametre1, parametre2):  
  
    """Docstring: Description de ce que fait la fonction.  
    Args:  
        parametre1: Description du premier paramètre.  
        parametre2: Description du second paramètre.  
    Returns:  
        Description de ce que la fonction retourne.  
    """  
  
    # Corps de la fonction  
    resultat = parametre1 + parametre2  
  
    return resultat # Retourne une valeur (optionnel)
```

Exemple simple : Calculer la puissance

Imaginons que nous voulions calculer la puissance électrique ($P = U * I$) pour différents équipements.

```
def calculer_puissance(tension, courant):  
  
    """Calcule la puissance électrique en Watts.  
  
    Args:  
        tension (float): La tension en Volts.  
        courant (float): Le courant en Ampères.  
  
    Returns:
```

```

    float: La puissance en Watts.
    """

    puissance = tension * courant
    return puissance

# Appel de la fonction
p_moteur = calculer_puissance(tension=230, courant=10)
print(f"Puissance du moteur : {p_moteur} Watts")

p_lampe = calculer_puissance(tension=12, courant=2)

print(f"Puissance de la lampe : {p_lampe} Watts")

```

La fonction return est utilisée pour renvoyer une valeur de la fonction. Si une fonction ne contient pas d'instruction return, elle renvoie implicitement None.

5.2 Portée des variables : Locales et globales

La **portée** d'une variable détermine où elle peut être accédée dans votre programme. [2]

- **Variables locales** : Définies à l'intérieur d'une fonction, elles ne sont accessibles qu'à l'intérieur de cette fonction.
- **Variables globales** : Définies en dehors de toute fonction, elles sont accessibles de n'importe où dans le programme.

```

# Variable globale

taux_efficacite_general = 0.95

def calculer_energie_utile(energie_totale):
    # energie_totale est une variable locale
    # taux_efficacite_general est une variable globale, accessible ici
    energie_utile = energie_totale * taux_efficacite_general
    return energie_utile

energie_fournie = 1000 # Joules
energie_recuperee = calculer_energie_utile(energie_fournie)
print(f"Énergie utile récupérée : {energie_recuperee} Joules")

# Tenter d'accéder à une variable locale en dehors de sa fonction (cela générera une erreur)

# print(energie_utile) # NameError: name 'energie_utile' is not defined

```

Il est généralement recommandé de minimiser l'utilisation de variables globales pour éviter les effets de bord inattendus et rendre le code plus facile à comprendre et à déboguer.

5.3 Arguments de fonction

Les fonctions peuvent accepter différents types d'arguments. [3]

Arguments par position

Les arguments sont passés à la fonction dans l'ordre où ils sont définis.

```
def afficher_info_capteur(id_capteur, type_capteur, localisation):  
  
    print(f"ID: {id_capteur}, Type: {type_capteur}, Localisation: {localisation}")  
  
afficher_info_capteur("C001", "Température", "Salle des machines")
```

Arguments par mot-clé

Vous pouvez spécifier les arguments en utilisant leur nom, ce qui rend le code plus lisible et permet de ne pas se soucier de l'ordre.

```
afficher_info_capteur(localisation="Atelier", id_capteur="C002", type_capteur="Pression")
```

Valeurs par défaut

Vous pouvez définir des valeurs par défaut pour les paramètres. Si l'argument n'est pas fourni lors de l'appel de la fonction, la valeur par défaut est utilisée.

```
def enregistrer_evenement(message, niveau="INFO", timestamp="maintenant"):  
  
    print(f"[{timestamp}] [{niveau}] {message}")  
  
enregistrer_evenement("Démarrage du système.")  
enregistrer_evenement("Erreur critique détectée !", niveau="CRITIQUE")  
  
enregistrer_evenement("Capteur déconnecté.", timestamp="2025-09-30 10:30:00")
```

5.4 Documentation des fonctions (Docstrings)

Les **docstrings** (chaînes de documentation) sont des chaînes de caractères qui apparaissent comme la première instruction dans une définition de fonction (ou de module, classe, méthode). Elles sont utilisées pour documenter le but de la fonction, ses arguments, et ce qu'elle retourne. [4]

Les docstrings sont importantes pour la lisibilité et la maintenabilité du code, surtout lorsque vous travaillez en équipe ou sur des projets complexes. Elles peuvent être consultées à l'aide de la fonction `help()` ou en accédant à l'attribut `__doc__` de la fonction.

```
def calculer_mtbf(temps_total_fonctionnement, nombre_pannes):  
  
    """Calcule le Temps Moyen Entre Pannes (MTBF).  
  
    Le MTBF est une mesure de la fiabilité d'un système ou d'un composant.  
    Il représente le temps moyen attendu entre deux pannes consécutives.  
  
    Args:  
        temps_total_fonctionnement (float): Le temps total pendant lequel le système a fonctionné (en heures).  
        nombre_pannes (int): Le nombre de pannes survenues pendant cette période.  
  
    Returns:  
        float: Le MTBF en heures. Retourne 0 si aucune panne n'est survenue pour éviter une division par zéro.  
    """  
    if nombre_pannes == 0:  
        return 0 # Ou gérer l'erreur autrement, selon le contexte  
    mtbf = temps_total_fonctionnement / nombre_pannes  
    return mtbf  
  
# Afficher la documentation de la fonction  
help(calculer_mtbf)  
  
# Utilisation de la fonction  
mtbf_pompe = calculer_mtbf(temps_total_fonctionnement=5000, nombre_pannes=2)  
print(f"MTBF de la pompe : {mtbf_pompe} heures")  
  
mtbf_capteur = calculer_mtbf(temps_total_fonctionnement=10000, nombre_pannes=0)  
  
print(f"MTBF du capteur (aucune panne) : {mtbf_capteur} heures")
```

Références

[1] Python.org. *The Python Tutorial - 4. More Control Flow Tools - 4.6. Defining Functions*. Disponible sur : <https://docs.python.org/3/tutorial/controlflow.html#defining-functions> [2]

Python.org. *The Python Tutorial* - 9. Classes - 9.2. *Python Scopes and Namespaces*. Disponible sur : <https://docs.python.org/3/tutorial/classes.html#python-scopes-and-namespaces> [3] Python.org. *The Python Tutorial* - 4. More Control Flow Tools - 4.7. *More on Defining Functions*. Disponible sur : <https://docs.python.org/3/tutorial/controlflow.html#more-on-defining-functions> [4] Python.org. *The Python Tutorial* - 4. More Control Flow Tools - 4.8. *Documentation Strings*. Disponible sur : <https://docs.python.org/3/tutorial/controlflow.html#documentation-strings>

Module 6 : Manipulation de fichiers et gestion des données

Dans ce module, nous allons apprendre à interagir avec le système de fichiers de votre ordinateur. La capacité de lire et d'écrire des données dans des fichiers est essentielle pour de nombreuses applications en maintenance électromécanique, comme l'enregistrement des relevés de capteurs, la gestion des historiques de maintenance, ou l'importation de configurations d'équipements. Nous aborderons l'ouverture, la lecture et l'écriture de fichiers, en nous concentrant sur les formats texte et CSV.

6.1 Ouverture et fermeture de fichiers

Pour travailler avec un fichier, la première étape est de l'ouvrir. L'ouverture d'un fichier crée un objet fichier que vous pouvez utiliser pour lire ou écrire. Une fois que vous avez terminé de travailler avec le fichier, il est crucial de le fermer pour libérer les ressources système. [1]

La fonction `open()`

La fonction `open()` prend au moins deux arguments : le nom du fichier (chemin) et le mode d'ouverture.

Modes d'ouverture courants :

- `'r'` : Lecture (par défaut). Le fichier doit exister.
- `'w'` : Écriture. Crée un nouveau fichier ou écrase un fichier existant.
- `'a'` : Ajout (append). Ajoute du contenu à la fin d'un fichier existant. Crée le fichier s'il n'existe pas.
- `'x'` : Création exclusive. Crée un nouveau fichier, mais échoue si le fichier existe déjà.
- `'t'` : Mode texte (par défaut). Les données sont traitées comme du texte.
- `'b'` : Mode binaire. Les données sont traitées comme des octets.

```
# Exemple d'ouverture et de fermeture manuelle
```

```
fichier = open("relevés_capteurs.txt", "w") # Ouvre en mode écriture
fichier.write("Température: 25.5°C\n")
fichier.write("Pression: 10.2 bar\n")
fichier.close() # Très important de fermer le fichier
```

```
print("Fichier 'releves_capteurs.txt' créé et écrit.")
```

L'instruction with open() as ... (recommandée)

La meilleure façon de travailler avec des fichiers en Python est d'utiliser l'instruction with. Elle garantit que le fichier est automatiquement fermé, même si des erreurs surviennent pendant son traitement. [1]

```
# Écriture avec 'with'

with open("historique_pannes.txt", "w") as f:
    f.write("2025-09-29, Moteur_A, Surchauffe\n")
    f.write("2025-09-30, Pompe_B, Fuite\n")

print("Fichier 'historique_pannes.txt' créé et écrit avec 'with'.")

# Ajout de contenu avec 'with'
with open("historique_pannes.txt", "a") as f:
    f.write("2025-10-01, Compresseur_C, Bruit anormal\n")

print("Contenu ajouté à 'historique_pannes.txt'.")
```

6.2 Lecture de fichiers

Une fois un fichier ouvert en mode lecture ('r'), vous pouvez en lire le contenu de différentes manières. [2]

read() : Lire tout le contenu

Lit l'intégralité du fichier sous forme d'une seule chaîne de caractères.

```
with open("releves_capteurs.txt", "r") as f:

    contenu = f.read()
    print("\nContenu de 'releves_capteurs.txt' (read()) :")

    print(contenu)
```

readline() : Lire ligne par ligne

Lit une seule ligne du fichier à la fois. Chaque appel à readline() lit la ligne suivante.

```
with open("historique_pannes.txt", "r") as f:

    print("\nContenu de 'historique_pannes.txt' (readline()) :")
    ligne1 = f.readline()
    ligne2 = f.readline()
    print(f"Première ligne : {ligne1.strip()}") # .strip() pour enlever les retours à la ligne

    print(f"Deuxième ligne : {ligne2.strip()}")
```

readlines() : Lire toutes les lignes dans une liste

Lit toutes les lignes du fichier et les retourne sous forme de liste de chaînes de caractères, où chaque élément de la liste est une ligne du fichier.

```
with open("historique_pannes.txt", "r") as f:

    lignes = f.readlines()
    print("\nContenu de 'historique_pannes.txt' (readlines()) :")
    for ligne in lignes:

        print(ligne.strip())
```

Itération directe sur l'objet fichier

La méthode la plus simple et la plus efficace pour lire un fichier ligne par ligne est d'itérer directement sur l'objet fichier dans une boucle for.

```
with open("historique_pannes.txt", "r") as f:

    print("\nContenu de 'historique_pannes.txt' (itération directe) :")
    for ligne in f:

        print(ligne.strip())
```

6.3 Écriture dans des fichiers

Nous avons déjà vu la méthode write() pour écrire des chaînes de caractères dans un fichier.
[3]

write() : Écrire une chaîne

Écrit la chaîne spécifiée dans le fichier. Vous devez ajouter manuellement les caractères de nouvelle ligne (n) si vous voulez que le texte soit sur plusieurs lignes.

```
with open("rapport_journalier.txt", "w") as f:

    f.write("Rapport du 2025-10-02\n")
    f.write("-----\n")
    f.write("Équipement A : OK\n")
    f.write("Équipement B : Alerte température\n")

print("Fichier 'rapport_journalier.txt' créé.")
```

writelines() : Écrire une liste de chaînes

Écrit une liste de chaînes de caractères dans le fichier. Chaque chaîne de la liste est écrite séquentiellement. Comme pour write(), vous devez inclure les caractères de nouvelle ligne dans les chaînes si vous souhaitez qu'elles apparaissent sur des lignes distinctes.

```
liste_alertes = [

    "Alerte 1: Pression basse Pompe_C\n",
    "Alerte 2: Vibration excessive Moteur_D\n",
    "Alerte 3: Niveau d'huile critique Compresseur_E\n"
]

with open("alertes_critiques.txt", "w") as f:
    f.writelines(liste_alertes)

print("Fichier 'alertes_critiques.txt' créé avec writelines.")
```

6.4 Formats courants : Fichiers texte (.txt) et CSV (.csv)

Fichiers texte (.txt)

Les fichiers .txt sont les plus simples. Ils stockent du texte brut. Nous les avons utilisés dans les exemples précédents.

Fichiers CSV (.csv)

Les fichiers **CSV (Comma Separated Values)** sont un format très courant pour stocker des données tabulaires (comme des feuilles de calcul ou des bases de données). Chaque ligne du fichier représente un enregistrement, et les valeurs de chaque enregistrement sont séparées par un délimiteur (souvent une virgule, mais cela peut être un point-virgule ou une tabulation). [4]

Python dispose d'un module `csv` intégré qui facilite la lecture et l'écriture de ces fichiers.

Écriture d'un fichier CSV

```
import csv

donnees_capteurs = [
    ["Capteur_ID", "Température", "Pression", "Date"],
    ["C001", 25.3, 10.2, "2025-09-29"],
    ["C002", 30.1, 12.5, "2025-09-29"],
    ["C001", 26.0, 10.3, "2025-09-30"]
]

with open("donnees_capteurs.csv", "w", newline="") as fichier_csv:
    writer = csv.writer(fichier_csv)
    writer.writerows(donnees_capteurs)

print("Fichier 'donnees_capteurs.csv' créé.")
```

- L'argument `newline=""` est important lors de l'ouverture de fichiers CSV pour éviter les lignes vides supplémentaires.
- `csv.writer()` crée un objet qui peut écrire des données CSV.
- `writer.writerows()` écrit plusieurs lignes à la fois à partir d'une liste de listes.

Lecture d'un fichier CSV

```
import csv

with open("donnees_capteurs.csv", "r") as fichier_csv:
    reader = csv.reader(fichier_csv)
    print("\nContenu de 'donnees_capteurs.csv' :")
    for ligne in reader:
        print(ligne)

# Lecture avec un DictReader pour accéder aux données par nom de colonne
with open("donnees_capteurs.csv", "r") as fichier_csv:
```

```
reader_dict = csv.DictReader(fichier_csv)
print("\nContenu de 'donnees_capteurs.csv' (DictReader) :")
for ligne in reader_dict:

    print(f"Capteur {ligne['Capteur_ID']} a enregistré {ligne['Température']}°C le {ligne['Date']}")
```

- `csv.reader()` crée un objet qui peut lire des données CSV.
- `csv.DictReader()` est très utile car il lit chaque ligne comme un dictionnaire, où les clés sont les en-têtes de colonne (la première ligne du fichier).

La manipulation de fichiers CSV est une compétence clé pour l'analyse de données en maintenance, car de nombreux systèmes exportent des données dans ce format.

Références

[1] Python.org. *Input and Output - Reading and Writing Files*. Disponible sur : <https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files> [2] Python.org. *Input and Output - Methods of File Objects*. Disponible sur : <https://docs.python.org/3/tutorial/inputoutput.html#methods-of-file-objects> [3] Python.org. *Input and Output - Methods of File Objects*. Disponible sur : <https://docs.python.org/3/tutorial/inputoutput.html#methods-of-file-objects> [4] Python.org. *csv — CSV File Reading and Writing*. Disponible sur : <https://docs.python.org/3/library/csv.html>

Module 7 : Gestion des erreurs et débogage

En programmation, les erreurs sont inévitables. Apprendre à les identifier, à les comprendre et à les corriger est une compétence essentielle pour tout développeur. Ce module vous guidera à travers les différents types d'erreurs en Python et vous montrera comment les gérer de manière élégante à l'aide de mécanismes d'exception, ainsi que des techniques de débogage simples.

7.1 Types d'erreurs

Il existe principalement deux catégories d'erreurs en programmation : les erreurs de syntaxe et les exceptions (erreurs d'exécution). [1]

Erreurs de syntaxe (SyntaxError)

Les erreurs de syntaxe sont les plus basiques. Elles se produisent lorsque le code ne respecte pas les règles grammaticales du langage Python. L'interpréteur Python ne peut même pas exécuter le programme s'il contient des erreurs de syntaxe.

```
# Exemple d'erreur de syntaxe
```

```
# print("Bonjour le monde" # Manque une parenthèse fermante
```

Lorsque vous exécutez ce code, Python affichera un SyntaxError et indiquera la ligne où l'erreur a été détectée, ce qui facilite la correction.

Exceptions (Erreurs d'exécution)

Les exceptions sont des erreurs qui se produisent pendant l'exécution du programme, même si la syntaxe est correcte. Elles interrompent le déroulement normal du programme si elles ne sont pas gérées. Python dispose de nombreux types d'exceptions intégrées. [2]

Voici quelques exemples courants :

- **NameError** : Se produit lorsque vous essayez d'utiliser une variable ou une fonction qui n'a pas été définie.

```
# print(variable_non_definie) # NameError: name 'variable_non_definie' is not defined
```

- **TypeError** : Se produit lorsque une opération est appliquée à un type de données inapproprié.

```
# "texte" + 5 # TypeError: can only concatenate str (not "int") to str
```

- **ZeroDivisionError** : Se produit lorsque vous tentez de diviser un nombre par zéro.

```
# 10 / 0 # ZeroDivisionError: division by zero
```

- **IndexError** : Se produit lorsque vous essayez d'accéder à un index qui est hors des limites d'une liste ou d'une autre séquence.

```
# ma_liste = [1, 2, 3]
```

```
# print(ma_liste[3]) # IndexError: list index out of range
```

- **FileNotFoundError** : Se produit lorsque vous essayez d'ouvrir un fichier qui n'existe pas.

```
# with open("fichier_inexistant.txt", "r") as f: # FileNotFoundError: [Errno 2] No such file or directory: 'fichier_inexistant.txt'
```

```
# pass
```

7.2 Gestion des exceptions (try, except, finally)

Python fournit un mécanisme robuste pour gérer les exceptions, ce qui permet à votre programme de continuer à s'exécuter même en cas d'erreur. Cela se fait à l'aide des blocs try, except et finally. [3]

Le bloc try-except

Le code susceptible de lever une exception est placé dans le bloc try. Si une exception se produit, l'exécution du bloc try est interrompue et le code du bloc except correspondant est exécuté. Si aucune exception ne se produit, le bloc except est ignoré.

```
def calculer_ratio_pannes(total_heures, nombre_pannes):

    try:
        ratio = total_heures / nombre_pannes
        print(f"Ratio pannes/heures : {ratio:.2f}")
    except ZeroDivisionError:
        print("Erreur : Le nombre de pannes ne peut pas être zéro pour calculer le ratio.")
        ratio = 0 # Valeur par défaut en cas d'erreur
    except TypeError:
        print("Erreur : Les entrées doivent être des nombres.")
        ratio = -1 # Indiquer une erreur avec une valeur spécifique
    except Exception as e: # Capture toute autre exception non spécifiée
        print(f"Une erreur inattendue s'est produite : {e}")
        ratio = -2
    return ratio

print("\n--- Test 1 : Division normale ---")
calculer_ratio_pannes(1000, 5)

print("\n--- Test 2 : Division par zéro ---")
calculer_ratio_pannes(1000, 0)

print("\n--- Test 3 : Erreur de type ---")
calculer_ratio_pannes("mille", 5)

print("\n--- Test 4 : Autre erreur (exemple) ---")
# Simuler une autre erreur pour l'exemple
def func_qui_leve_erreur():
    raise ValueError("Ceci est une erreur de valeur simulée")

try:
    func_qui_leve_erreur()
except Exception as e:
```

```
print(f"Capture de l'erreur simulée : {e}")
```

Vous pouvez spécifier plusieurs blocs except pour gérer différents types d'exceptions. Il est généralement une bonne pratique de capturer les exceptions les plus spécifiques en premier, puis les plus générales.

Le bloc finally

Le bloc finally est optionnel et son code est **toujours exécuté**, que l'exception se soit produite ou non, et qu'elle ait été gérée ou non. Il est souvent utilisé pour des opérations de nettoyage, comme la fermeture de fichiers ou la libération de ressources. [4]

```
import os

def traiter_fichier_config(nom_fichier):
    fichier = None
    try:
        fichier = open(nom_fichier, "r")
        contenu = fichier.read()
        print(f"Contenu du fichier {nom_fichier} :\n{contenu}")
    except FileNotFoundError:
        print(f"Erreur : Le fichier {nom_fichier} n'a pas été trouvé.")
    except Exception as e:
        print(f"Une erreur inattendue s'est produite lors du traitement du fichier : {e}")
    finally:
        if fichier:
            fichier.close()
            print(f"Le fichier {nom_fichier} a été fermé.")

# Créer un fichier temporaire pour le test
with open("config.txt", "w") as f:
    f.write("param1=valeur1\nparam2=valeur2")

print("\n--- Test 1 : Fichier existant ---")
traiter_fichier_config("config.txt")

print("\n--- Test 2 : Fichier inexistant ---")
traiter_fichier_config("fichier_non_existant.txt")

# Nettoyer le fichier temporaire

os.remove("config.txt")
```

7.3 Débogage simple

Le débogage est le processus de recherche et de correction des erreurs (bugs) dans votre code. Pour les débutants, l'utilisation de la fonction `print()` est une méthode simple et efficace pour suivre l'exécution de votre programme et inspecter les valeurs des variables. [5]

Utilisation de `print()` pour le débogage

En insérant des instructions `print()` à des points stratégiques de votre code, vous pouvez :

- **Vérifier l'exécution** : Confirmer qu'une partie du code est atteinte.
- **Inspecter les valeurs des variables** : Voir ce que contiennent les variables à un moment donné.
- **Suivre le flux du programme** : Comprendre l'ordre dans lequel les instructions sont exécutées.

```
def calculer_temperature_moyenne(relevés_temperatures):

    print(f"[DEBUG] Début de la fonction. Relevés reçus : {relevés_temperatures}")
    if not relevés_temperatures:
        print("[DEBUG] Liste de relevés vide. Retourne 0.")
        return 0

    somme_temperatures = 0
    for temp in relevés_temperatures:
        somme_temperatures += temp
        print(f"[DEBUG] Température actuelle : {temp}, Somme cumulée : {somme_temperatures}")

    moyenne = somme_temperatures / len(relevés_temperatures)
    print(f"[DEBUG] Somme : {somme_temperatures}, Nombre de relevés : {len(relevés_temperatures)}, Moyenne calculée : {moyenne}")
    return moyenne

print("\n--- Calcul de la moyenne 1 ---")
capteur_data_1 = [22.5, 23.1, 22.9, 23.0]
resultat_1 = calculer_temperature_moyenne(capteur_data_1)
print(f"Moyenne finale : {resultat_1:.2f}°C")

print("\n--- Calcul de la moyenne 2 (liste vide) ---")
capteur_data_2 = []
resultat_2 = calculer_temperature_moyenne(capteur_data_2)

print(f"Moyenne finale : {resultat_2:.2f}°C")
```

Messages d'erreur Python

Lorsque Python rencontre une exception non gérée, il affiche un **traceback** (pile d'appels). C'est une information très précieuse qui indique :

- Le type d'erreur (NameError, TypeError, etc.).
- Le message d'erreur détaillé.
- La séquence d'appels de fonctions qui a mené à l'erreur, avec les numéros de ligne. Cela vous aide à localiser précisément l'origine du problème.

Apprenez à lire et à interpréter ces messages d'erreur. Ils sont vos meilleurs alliés pour le débogage.

Références

[1] Python.org. *The Python Tutorial - 8. Errors and Exceptions*. Disponible sur : <https://docs.python.org/3/tutorial/errors.html> [2] Python.org. *Built-in Exceptions*. Disponible sur : <https://docs.python.org/3/library/exceptions.html> [3] Python.org. *The Python Tutorial - 8. Errors and Exceptions - 8.3. Handling Exceptions*. Disponible sur : <https://docs.python.org/3/tutorial/errors.html#handling-exceptions> [4] Python.org. *The Python Tutorial - 8. Errors and Exceptions - 8.5. Defining Clean-up Actions*. Disponible sur : <https://docs.python.org/3/tutorial/errors.html#defining-clean-up-actions> [5] Real Python. *Python Debugging With Pdb*. Disponible sur : <https://realpython.com/python-debugging-pdb/> (Bien que cet article parle de pdb, la section sur l'approche générale du débogage est pertinente).

Module 8 : Exercices appliqués à la maintenance électromécanique

Ce module est dédié à l'application pratique des concepts de programmation Python que vous avez appris, spécifiquement dans le contexte de la maintenance électromécanique. L'objectif est de vous montrer comment Python peut être un outil puissant pour résoudre des problèmes réels dans votre domaine, de l'analyse de données de capteurs à la planification de maintenance.

Chaque exercice sera présenté avec un énoncé clair, des données d'exemple si nécessaire, et des indications sur les concepts Python à utiliser. Il est fortement recommandé de tenter de résoudre ces exercices par vous-même avant de consulter d'éventuelles solutions (qui ne seront pas fournies ici, l'objectif étant la réflexion et la pratique).

8.1 Analyse de données de capteurs

Les capteurs sont omniprésents dans les systèmes électromécaniques modernes. L'analyse de leurs données est cruciale pour la maintenance prédictive et la détection d'anomalies.

Exercice 8.1.1 : Calcul de la température moyenne et détection de seuil

Vous disposez d'un fichier `temperatures.csv` contenant des relevés de température d'un moteur sur une période donnée. Le fichier a le format suivant :

Timestamp, Temperature_C
2025-09-30 08:00:00, 75.2
2025-09-30 08:05:00, 76.1
2025-09-30 08:10:00, 77.5
2025-09-30 08:15:00, 78.9
2025-09-30 08:20:00, 80.3
2025-09-30 08:25:00, 81.5
2025-09-30 08:30:00, 82.8
2025-09-30 08:35:00, 84.1
2025-09-30 08:40:00, 85.5
2025-09-30 08:45:00, 86.8
2025-09-30 08:50:00, 88.2
2025-09-30 08:55:00, 89.5
2025-09-30 09:00:00, 91.0

Tâches :

- 11 Écrivez un script Python qui lit ce fichier `temperatures.csv`.
- 12 Calculez la température moyenne enregistrée.
- 13 Définissez un seuil d'alerte (par exemple, 85°C).
- 14 Identifiez et affichez tous les relevés où la température dépasse ce seuil, en indiquant le `Timestamp` et la `Temperature_C` correspondante.
- 15 Utilisez une fonction pour encapsuler le calcul de la moyenne et une autre pour la détection de seuil.
- 16 Gérez les erreurs potentielles (par exemple, fichier non trouvé, données non numériques).

Concepts Python à utiliser : Lecture de fichiers CSV (`csv` module), listes, boucles `for`, conditions `if`, fonctions, gestion des erreurs `try-except`.

Exercice 8.1.2 : Surveillance de l'état d'un équipement avec plusieurs capteurs

Vous avez des données de plusieurs capteurs (température, pression, vibration) pour un équipement. Ces données sont stockées dans un dictionnaire Python, où chaque clé est le nom du capteur et la valeur est une liste de ses relevés.

```

donnees_equipement = {

    "Temperature": [78.5, 79.1, 80.2, 81.5, 82.0, 83.5, 84.0, 85.1, 86.2, 87.5],
    "Pression": [5.2, 5.3, 5.1, 5.4, 5.5, 5.6, 5.7, 5.8, 5.9, 6.0],
    "Vibration": [0.15, 0.16, 0.17, 0.18, 0.19, 0.20, 0.21, 0.22, 0.23, 0.24]
}

seuils_alerte = {
    "Temperature": {"min": 70.0, "max": 85.0},
    "Pression": {"min": 4.5, "max": 5.8},
    "Vibration": {"min": 0.0, "max": 0.22}
}

```

Tâches :

- 17 Écrivez une fonction qui prend en entrée les `donnees_equipement` et les `seuils_alerte`.
- 18 Pour chaque capteur, vérifiez si la dernière valeur enregistrée dépasse les seuils min ou max définis.
- 19 Affichez un message d'alerte détaillé pour chaque capteur dont la valeur est hors de la plage normale, indiquant le nom du capteur, la valeur actuelle, et la plage acceptable.
- 20 Si toutes les valeurs sont dans la plage, affichez un message indiquant que l'équipement est en bon état.

Concepts Python à utiliser : Dictionnaires, boucles `for`, conditions `if-elif-else`, fonctions.

8.2 Planification de maintenance

La planification de maintenance est essentielle pour optimiser la durée de vie des équipements et minimiser les temps d'arrêt.

Exercice 8.2.1 : Calcul de la fréquence de maintenance

Vous avez une liste d'équipements, chacun avec un nombre d'heures de fonctionnement et une fréquence de maintenance recommandée en heures.

```

equipements_maintenance = [

    {"Nom": "Moteur Principal", "Heures_Fonctionnement": 1250, "Frequence_Maintenance": 1000},
    {"Nom": "Pompe de Refroidissement", "Heures_Fonctionnement": 800, "Frequence_Maintenance": 1500},
    {"Nom": "Compresseur d'Air", "Heures_Fonctionnement": 2100, "Frequence_Maintenance": 2000},
    {"Nom": "Ventilateur d'Extraction", "Heures_Fonctionnement": 300, "Frequence_Maintenance": 500}
]

```

```
] ]
```

Tâches :

- 21 Écrivez une fonction qui prend cette liste d'équipements en entrée.
- 22 Pour chaque équipement, calculez le nombre d'heures restantes avant la prochaine maintenance.
- 23 Si un équipement a dépassé sa fréquence de maintenance, affichez un message d'alerte spécifique.
- 24 Affichez un rapport clair indiquant le nom de l'équipement, les heures de fonctionnement actuelles, la fréquence de maintenance et les heures restantes (ou le dépassement).

Concepts Python à utiliser : Listes de dictionnaires, boucles for, conditions if-else, fonctions, formatage de chaînes.

Exercice 8.2.2 : Génération d'un planning de maintenance simple

Créez un programme qui génère un planning de maintenance hebdomadaire pour une série de tâches. Chaque tâche a un nom et un jour de la semaine où elle doit être effectuée.

```
taches_maintenance = [  
    {"Tache": "Inspection visuelle Moteur_A", "Jour": "Lundi"},  
    {"Tache": "Graissage Pompe_B", "Jour": "Mercredi"},  
    {"Tache": "Vérification filtres Compresseur_C", "Jour": "Lundi"},  
    {"Tache": "Nettoyage Ventilateur_D", "Jour": "Vendredi"},  
    {"Tache": "Contrôle niveaux Huile", "Jour": "Mercredi"}  
]
```

Tâches :

- 25 Organisez ces tâches par jour de la semaine.
- 26 Affichez le planning de maintenance pour la semaine, en listant les tâches pour chaque jour (du Lundi au Vendredi).
- 27 Gérez le cas où aucun jour n'est spécifié ou si le jour est invalide.

Concepts Python à utiliser : Dictionnaires, listes, boucles for, conditions if-elif-else, tri (optionnel).

8.3 Simulation simple

La simulation peut aider à comprendre le comportement des systèmes et à tester des scénarios.

Exercice 8.3.1 : Modélisation de la dégradation d'un composant

Un composant a une durée de vie nominale de 1000 heures. Chaque heure de fonctionnement, il y a une petite probabilité qu'il se dégrade. Simulez la durée de vie de ce composant.

Tâches :

- 28 Définissez une probabilité de défaillance par heure (par exemple, 0.1% ou 0.001).
- 29 Simulez le fonctionnement du composant heure par heure.
- 30 Arrêtez la simulation lorsque le composant tombe en panne (utilisez un nombre aléatoire pour déterminer la panne).
- 31 Affichez le nombre total d'heures de fonctionnement avant la panne.
- 32 Répétez la simulation plusieurs fois (par exemple, 100 fois) et calculez la durée de vie moyenne du composant.

Concepts Python à utiliser : Boucles while, nombres aléatoires (random module), conditions if, variables, fonctions.

8.4 Rapports automatisés

La génération de rapports est une tâche courante qui peut être automatisée pour gagner du temps et assurer la cohérence.

Exercice 8.4.1 : Génération d'un rapport de maintenance simple

À partir des données de l'exercice 8.2.1 (liste d'équipements avec heures de fonctionnement et fréquence de maintenance), générez un rapport de maintenance au format texte.

Tâches :

- 33 Écrivez une fonction qui prend la liste des équipements et génère une chaîne de caractères formatée comme un rapport.
- 34 Le rapport doit inclure un titre, la date de génération, et pour chaque équipement, son nom, ses heures de fonctionnement, sa fréquence de maintenance et les heures restantes/dépassement.
- 35 Écrivez ce rapport dans un fichier rapport_maintenance_quotidien.txt.

Concepts Python à utiliser : Fonctions, formatage de chaînes, écriture de fichiers texte.

Ces exercices sont conçus pour être progressifs et pour vous encourager à combiner les différents concepts appris dans les modules précédents. N'hésitez pas à expérimenter et à adapter les problèmes à des scénarios plus spécifiques que vous rencontrez dans votre travail. Bonne pratique !